

Relatório de Laboratório — Lab02

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab02 — Assistentes de IA vs. Codificação Manual: um experimento controlado
Grupo (trio)	Jonas Martins · Caio Victor Kodato Teixeira · Henrique Volponi
Link do repositório / GitHub Projects	https://github.com/CaioVKodato/lab-experimentacao-de-software-2
Data de entrega	24/09/2026

1. Introdução

Ferramentas de IA generativa aplicadas à programação — GitHub Copilot, ChatGPT, Claude, Gemini, entre outras — tornaram-se onipresentes no dia a dia do desenvolvimento de software nos últimos anos, mas a maior parte do que se discute sobre seu real impacto em produtividade e qualidade ainda é relato anedótico, sem controle experimental. Esse tipo de evidência é insuficiente para orientar decisões técnicas ou de processo em equipes reais: sem medição controlada, não é possível distinguir um ganho genuíno de produtividade de um viés de confirmação de quem já está predisposto a gostar da ferramenta. Este laboratório busca preencher essa lacuna com um experimento controlado, comparando diretamente o uso de um assistente de IA generativa contra a codificação manual na resolução de tarefas de programação equivalentes.

O trabalho foi estruturado em torno de três Questões de Pesquisa (RQs), seguindo o método GQM (Goal-Question-Metric):

RQ1. O uso de assistente de IA reduz o tempo necessário para resolver uma tarefa de programação?

RQ2. O uso de assistente de IA reduz a quantidade de defeitos (testes que falham) no código produzido?

RQ3. O uso de assistente de IA altera a complexidade ciclomática ou a duplicação do código produzido?

Hipóteses informais do grupo, formuladas antes da coleta de dados:

RQ1 — esperava-se que o uso de IA reduzisse o tempo de resolução, dado que grande parte do esforço de codificação (escrever a sintaxe, lembrar de assinaturas de função, tratar casos-limite óbvios) é delegada ao assistente.

RQ2 — esperava-se uma taxa de sucesso levemente maior com IA, assumindo que o assistente cometeria menos erros de sintaxe e de lógica básica do que um desenvolvedor sob pressão de tempo.

RQ3 — não havia uma hipótese direcional forte; era plausível tanto que código gerado por IA fosse mais verboso e complexo (por gerar soluções mais defensivas/genéricas) quanto mais enxuto (por seguir padrões consistentes).

Além das três RQs do enunciado, o grupo propôs uma investigação adicional como inovação (os 30% de contribuição própria): o número de prompts necessários até a IA convergir para uma solução correta varia conforme a natureza da informação omitida do enunciado do problema — regra de negócio arbitrária versus convenção geral de boa prática de programação — e conforme o grau de organização estrutural do prompt usado para descrever o problema. O detalhamento da metodologia dessa investigação está na Seção 3.6.

2. Contexto

Este é o segundo laboratório da disciplina (Lab02), situado logo após o Lab01 (mineração de repositórios populares do GitHub e setup inicial do Kanban do grupo). Diferente do Lab01, que era um estudo observacional sobre dados públicos já existentes, o Lab02 é um experimento controlado conduzido pelo próprio grupo: os três integrantes são simultaneamente os pesquisadores e os sujeitos experimentais, resolvendo pessoalmente os katas sob as duas condições (com e sem IA).

O objeto de estudo é o processo de resolução de seis katas de programação — pequenos exercícios autocontidos, com enunciado e testes de aceitação bem definidos, escolhidos por dificuldade equivalente e baixa indexação pública (para reduzir o risco de a IA reproduzir uma solução memorizada de seu treinamento em vez de efetivamente raciocinar sobre o problema apresentado). Cada um dos três integrantes resolveu os seis katas individualmente, três com o assistente de IA habilitado e três sem, em um desenho experimental do tipo crossover / within-subject contrabalançado: cada pessoa serve como seu próprio controle, e a atribuição de quais katas cada integrante resolve em qual tratamento foi distribuída de forma que nenhum kata específico ficasse sempre associado ao mesmo tratamento entre os três integrantes.

Como referencial teórico para o desenho do experimento, o grupo adotou o método GQM (Goal-Question-Metric), de Basili, Caldiera e Rombach — a estrutura que organiza as RQs do enunciado como as "Questions" derivadas de um "Goal" único (analisar o efeito do assistente de IA sobre tempo, qualidade funcional e qualidade estrutural do código), com métricas candidatas específicas associadas a cada uma. A ferramenta de métricas estáticas escolhida (Radon) e a linguagem de programação (Python) foram decisões conjuntas — o enunciado exige Java caso se opte por CK, e Radon caso se opte por Python; o grupo optou por Python por menor curva de configuração do ambiente entre os três integrantes.

3. Metodologia

3.1 Principais Desafios

O principal desafio metodológico foi padronizar katas de dificuldade equivalente que fossem, ao mesmo tempo, pouco indexados publicamente — reduzindo o risco de o assistente de IA reproduzir uma solução já memorizada em seu treinamento em vez de efetivamente raciocinar sobre o problema apresentado. Isso levou o grupo a optar por seis katas inteiramente autorais, em vez de reaproveitar exercícios clássicos de plataformas como LeetCode ou HackerRank.

Um segundo desafio, específico da investigação de inovação do grupo (Seção 3.6), surgiu apenas durante a execução: a primeira tentativa de desenhar um kata com uma regra de negócio deliberadamente omitida do enunciado mostrou-se metodologicamente falha, porque a regra escolhida — normalização de maiúsculas/minúsculas em comparação de string — coincidia com uma convenção defensiva de programação que o assistente de IA aplicou por hábito, independentemente de qualquer pista no enunciado. Isso exigiu um redesenho, substituindo a regra escondida por políticas de negócio genuinamente arbitrárias, o que se mostrou necessário para isolar corretamente o efeito pretendido.

Por fim, como em qualquer laboratório desta disciplina, o GitHub Projects não expõe histórico de mudança de coluna consultável via API, o que exigiu snapshots manuais do Kanban ao final de cada sprint para preservar rastreabilidade do fluxo de trabalho real do grupo.

3.2 Tomadas de Decisão

Limite de WIP: a coluna Doing do board foi limitada a 4 cartões simultâneos, justificado pela janela semanal de trabalho do grupo (sprints executadas de segunda a quinta-feira — 4 dias úteis), limitando o WIP ao mesmo número de dias disponíveis.

Assistente de IA: o grupo padronizou o uso do Claude (versão gratuita) em todos os trials com IA habilitada, dos três integrantes, garantindo que o tratamento fosse comparável dentro do experimento.

Linguagem de programação: Python foi escolhido em vez de Java, determinando o uso do Radon (em vez do CK) como ferramenta de métricas estáticas.

Quantidade de katas: o grupo optou por 6 katas (3 com IA, 3 sem, por integrante) em vez do mínimo de 4 permitido pelo enunciado, ampliando o volume de trials (18 no total) para dar mais robustez à análise estatística.

Divisão de tratamentos: a atribuição de quais katas cada integrante resolve com ou sem IA foi definida de forma contrabalançada, evitando que uma eventual diferença de dificuldade real entre katas se confundisse com o efeito do tratamento.

3.3 Etapas

O processo de desenvolvimento seguiu a estrutura de sprints definida pelo enunciado, com entregas rastreadas via Issues no GitHub Projects, refletindo os Assignees reais registrados no board.

Sprint	Entregas	Responsável(is)	Issues (n°)
Lab02S01	Estrutura do repositório, labels e Kanban	Caio	#1
Lab02S01	Script de cronometragem e coleta de tempo	Caio	#2
Lab02S01	Ambiente e script de métricas estáticas (Radon/jscpd)	Henrique	#3
Lab02S01	Seleção e validação dos 6 katas	Jonas	#4
Lab02S01	Hipóteses, GQM e ameaças à validade	Jonas	#5
Lab02S01	Snapshot do Kanban Lab02S01	Caio	#6
Lab02S02	Trials k2, k4, k6 (IA) + k1, k3, k5 (Manual)	Caio	#11–#16
Lab02S02	Trials k1, k4, k5 (IA) + k2, k3, k6 (Manual)	Henrique	#17–#22
Lab02S02	Trials k1, k3, k5 (IA) + k2, k4, k6 (Manual)	Jonas	#23–#28
Lab02S03	Consolidação de métricas estruturais (LOC + duplicação)	Caio	#38
Lab02S03	RQ3: análise de estrutura do código	Caio	#40
Lab02S03	Dashboard de visualização (Pandas/Matplotlib/Seaborn)	Jonas	#41
Lab02S03	RQ1/RQ2: testes estatísticos (Wilcoxon/Mann-Whitney)	Henrique	#47
Lab02S03	RQ extra: ambiguidade e desorganização de prompt (k7, k8, k9)	Jonas	#48
Relatório Final	Elaboração do documento final	Jonas	#49

Configuração do processo

O board (GitHub Projects v2) foi estruturado com cinco colunas — Backlog → To Do → Doing → Review → Done —, mantendo o mesmo fluxo adotado no Lab01. O limite de WIP definido para a coluna Doing foi de 4 cartões simultâneos, justificado pela janela semanal de trabalho do grupo: as sprints são executadas de segunda a quinta-feira (4 dias úteis).

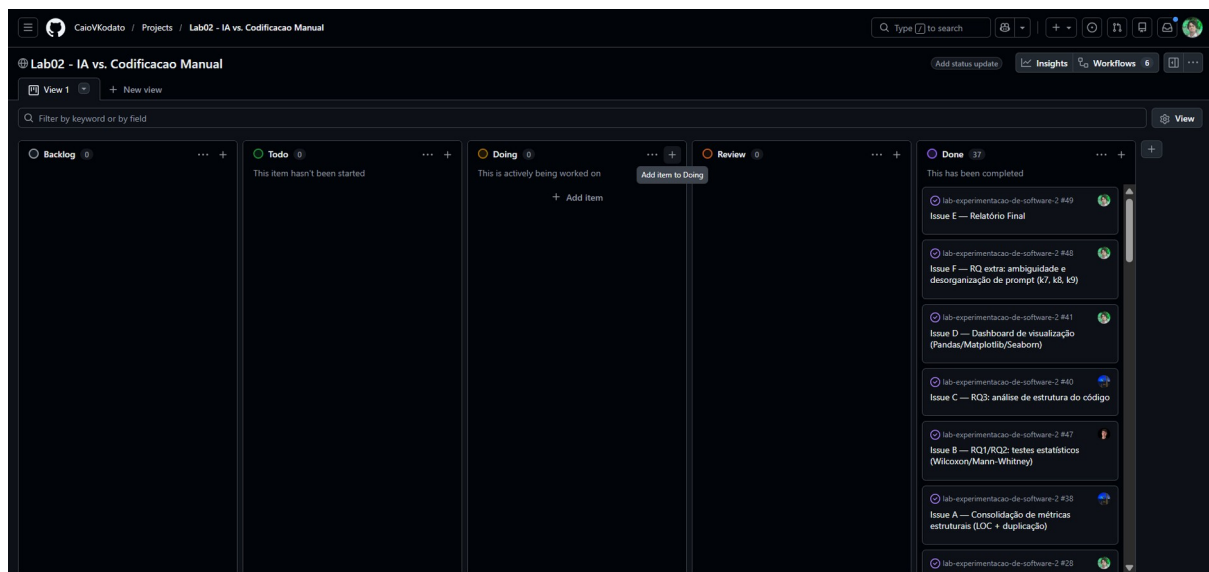


Figura 0 — Board final do LAB02 (GitHub Projects): 37 itens concluídos em Done, colunas Backlog/To Do/Doing/Review vazias, refletindo o fluxo de trabalho completo do grupo.

3.4 Ferramentas

- Python 3.13 como linguagem de implementação dos katas e dos scripts de análise.
- pytest para os testes de aceitação de cada kata.
- Radon (cc e mi) para métricas de complexidade ciclomática e índice de manutenibilidade.
- jscpd para percentual de duplicação de código — RQ3.
- Claude (versão gratuita, Anthropic) como assistente de IA nos trials com tratamento IA.
- Gemini (Google), usado exclusivamente na investigação de inovação do grupo, para evitar contaminação entre quem desenhou os katas de ambiguidade e quem tentou resolvê-los.
- Pandas para manipulação e consolidação dos dados de trials.
- SciPy (scipy.stats) para os testes estatísticos — Wilcoxon (pareado) e Mann-Whitney (não pareado), com tamanho de efeito.
- Matplotlib e Seaborn para visualização dos resultados.
- GitHub Projects (v2) como ferramenta de processo.

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta/Fonte
RQ1	Time-to-green	Tempo desde o início do trial até todos os testes de aceitação passarem; censurado em 35 min se não concluído	Segundos	Cronometragem manual
RQ2	Taxa de sucesso	Testes passando / testes totais ao final do time-box	Proporção (0–1)	pytest
RQ3	Complexidade ciclomática	Complexidade média por função (Radon cc)	Número (média)	Radon

RQ3	Duplicação de código	% de linhas duplicadas	Percentual	jscpd
RQ3 (controle)	LOC	Linhas de código do solucao.py final do trial	Contagem	Radon raw
RQ extra	Nº de prompts até convergir	Nº de interações com a IA até passar em todos os testes	Contagem	Registro manual

3.6 Inovações Propostas pelo Grupo (30% da nota)

A inovação do grupo consiste em uma RQ extra própria, investigando se o número de prompts necessários até a IA convergir para uma solução correta varia conforme dois fatores: (a) a natureza da informação omitida do enunciado — uma regra de negócio arbitrária versus uma convenção geral de boa prática de programação — e (b) o grau de organização estrutural do prompt usado para descrever o problema.

Foram desenhados três katas adicionais (k7, k8, k9), cada um com uma regra de negócio deliberadamente omitida do enunciado e revelada apenas pelos testes automatizados. O k7 usa uma regra que coincide com hábito defensivo de programação (normalização de maiúsculas/minúsculas); k8 e k9 usam regras de negócio genuinamente arbitrárias (carência de dias para um tipo específico de usuário; duplicação de pontos em um dia da semana específico). Os três enunciados foram apresentados ao Gemini em registro informal e estruturalmente desorganizado, e o k7 foi replicado em uma segunda execução independente para verificar consistência do resultado. O resultado desta investigação é discutido em detalhe na Seção 4.3.

4. Resultados

4.1 Coleta de Dados

A amostra final consiste em 18 trials válidos (data/trials.csv): 3 integrantes $\times$ 6 katas cada, sendo 9 trials no tratamento IA e 9 no tratamento Manual, distribuídos de forma contrabalançada. Todos os 18 trials foram concluídos dentro do time-box de 35 minutos — nenhum trial precisou ser registrado como censurado.

As métricas de tempo e taxa de sucesso foram registradas manualmente por cada integrante durante a execução de seus próprios trials (Lab02S02). As métricas estruturais — LOC, complexidade ciclomática média e percentual de duplicação — foram coletadas posteriormente, de forma centralizada, sobre o código final de cada um dos 18 trials (Lab02S03). Não foi aplicado nenhum critério de exclusão de outliers sobre os dados de tempo ou complexidade — dado o tamanho amostral já reduzido ($n=9$ por tratamento), o grupo optou por reportar todas as observações.

Durante a consolidação dos dados, foi identificado e corrigido um erro de rotulagem: os registros de um dos integrantes haviam sido inicialmente lançados com o tratamento invertido em relação à divisão de contrabalanceamento real. O erro foi detectado por conferência cruzada e corrigido antes de qualquer análise estatística ser finalizada — nenhum resultado reportado neste relatório foi calculado sobre o dado incorreto.

4.2 Visualização Gráfica

RQ1 — O uso de assistente de IA reduz o tempo necessário para resolver uma tarefa de programação? Mediana: 180s (IA) vs. 780s (Manual).

RQ2 — O uso de assistente de IA reduz a quantidade de defeitos no código produzido? Taxa de sucesso de 100% em ambos os tratamentos, sem variância.

RQ3 — O uso de assistente de IA altera a complexidade ciclomática ou a duplicação do código produzido? Complexidade mediana: 4,0 (IA) vs. 6,0 (Manual); duplicação: 0% em ambos.

Dashboard S03 — LAB02: IA vs. Codificação Manual

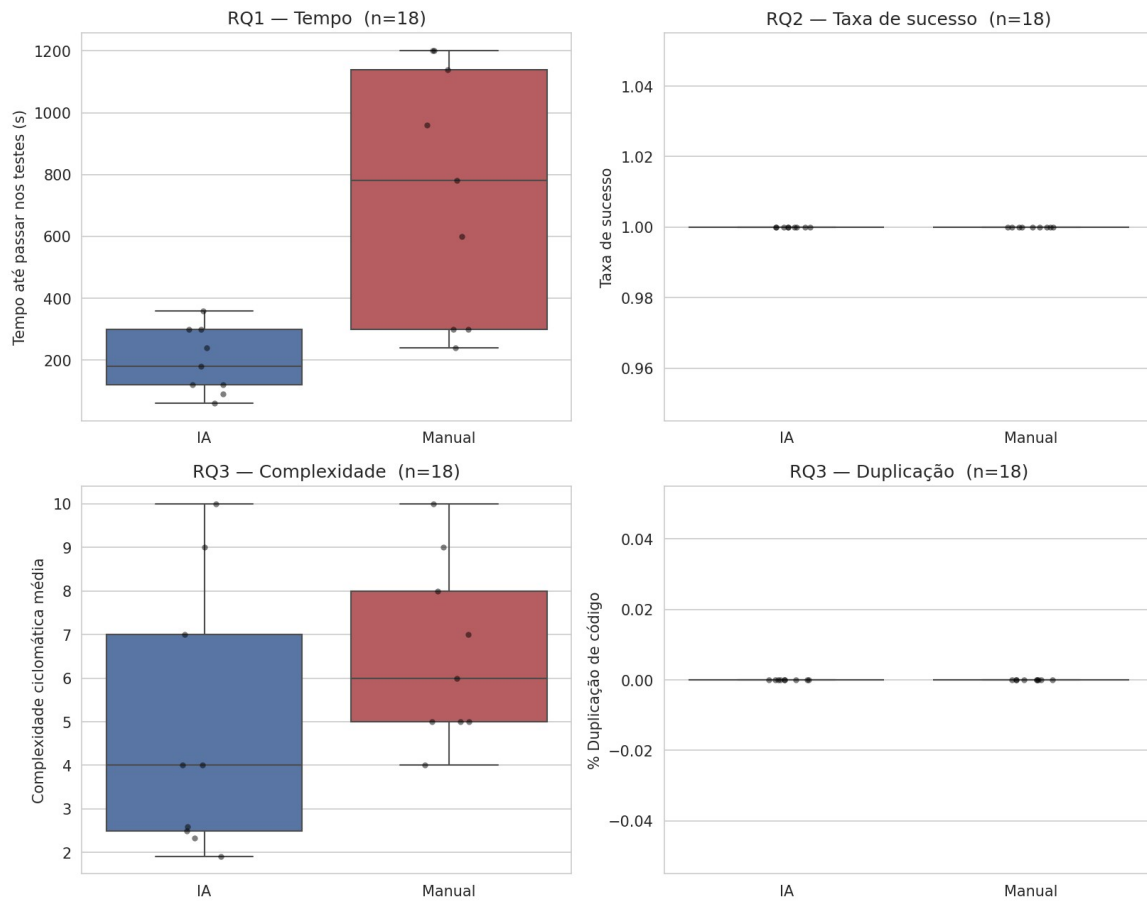


Figura 1 — Dashboard consolidado: RQ1 (tempo), RQ2 (taxa de sucesso), RQ3 (complexidade e duplicação), comparando os tratamentos IA e Manual (n=9 por grupo).

Duas visualizações complementares aprofundam a relação entre tempo e complexidade, e a distribuição geral do tempo entre os 18 trials.

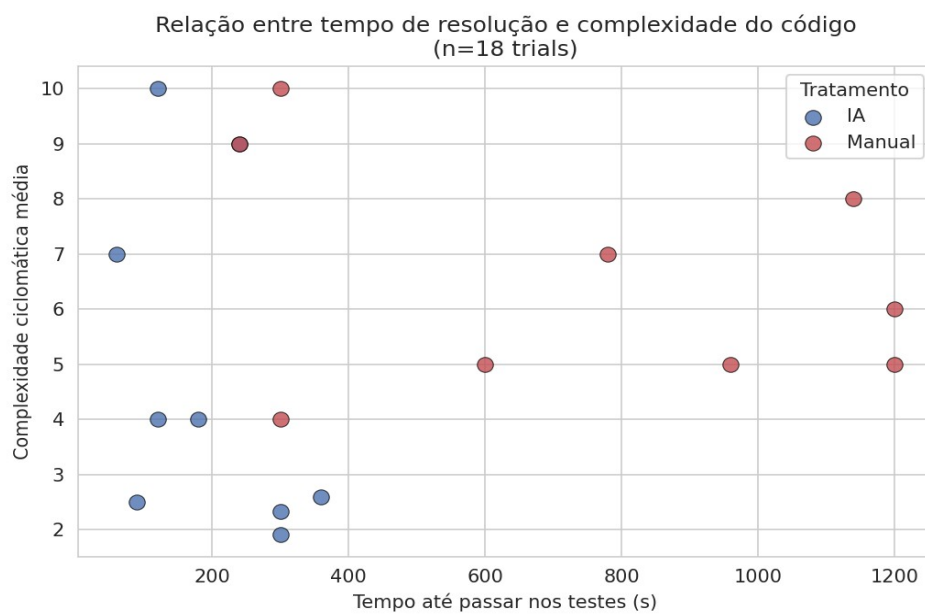


Figura 2 — Dispersão entre tempo de resolução e complexidade ciclomática média, por trial (n=18). Os pontos do tratamento IA (azul) concentram-se no quadrante inferior esquerdo (mais rápidos e menos complexos); os do tratamento Manual (vermelho) espalham-se em direção a tempos e complexidades maiores.

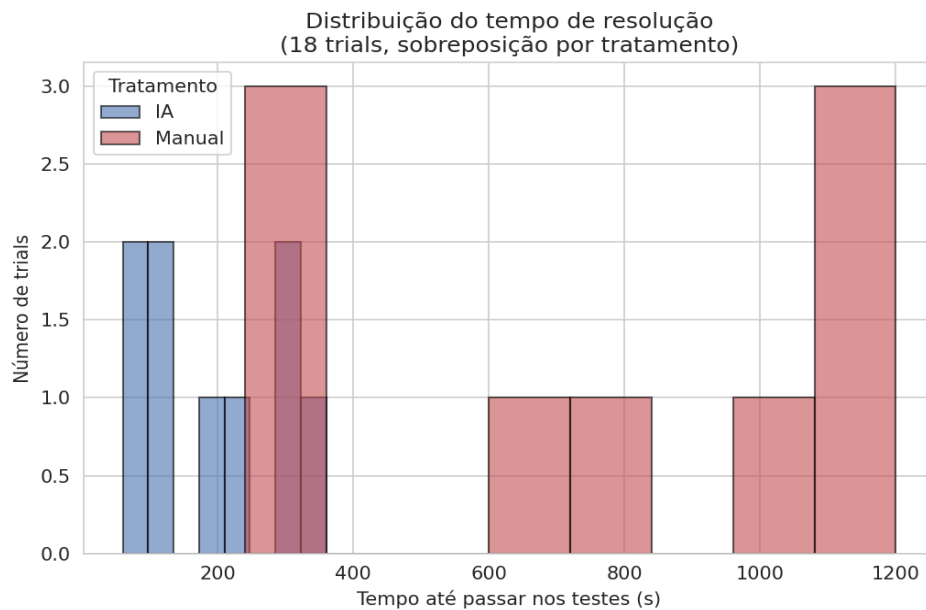


Figura 3 — Distribuição do tempo de resolução nos 18 trials, com sobreposição por tratamento. Os trials com IA concentram-se abaixo de 250s; os trials Manual se espalham por uma faixa bem mais ampla, incluindo os quatro trials acima de 900s.

Dois visualizações adicionais relacionam tamanho e complexidade do código: o volume de linhas produzido em cada trial, e a complexidade normalizada por esse volume.

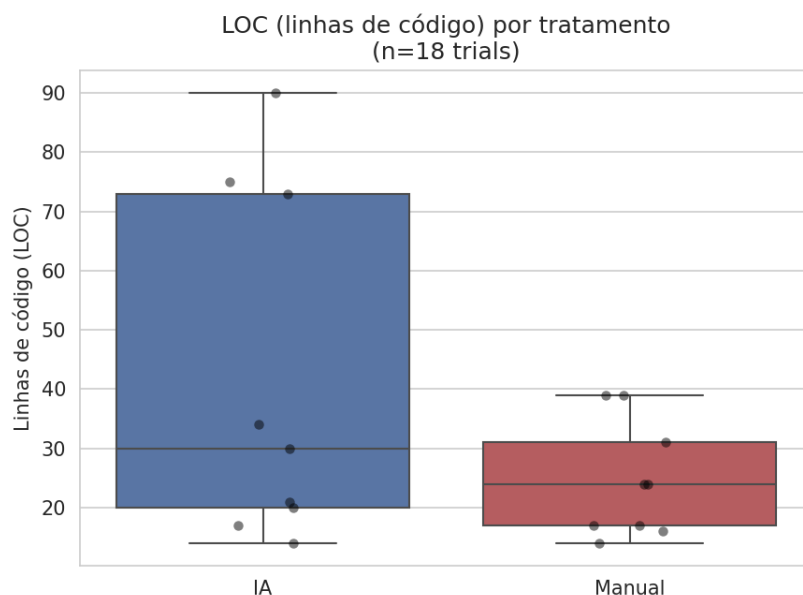


Figura 4 — LOC (linhas de código) por tratamento (n=18). A mediana com IA (30 linhas) é maior que com Manual (24 linhas), com maior dispersão — três trials com IA ultrapassam 70 linhas, acima de qualquer trial Manual.

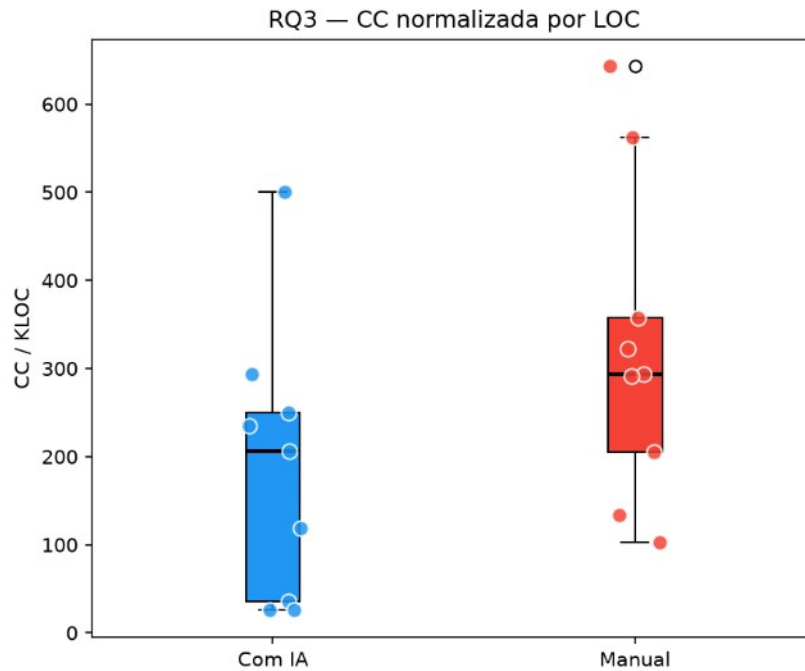


Figura 5 — Complexidade ciclomática normalizada por mil linhas de código (CC/KLOC), por tratamento. A separação entre os grupos fica mais nítida que na complexidade bruta (Figura 1): mediana de aproximadamente 200 (IA) contra 300 (Manual).

4.3 Discussão

RQ1 — Tempo até passar nos testes

Hipótese informal: esperava-se que o uso de IA reduzisse o tempo de resolução.

Resultado: a hipótese se confirma, com evidência estatística forte. A mediana do tempo até passar em todos os testes foi de 180 segundos com IA contra 780 segundos sem IA — uma diferença de aproximadamente 4,3 vezes, estatisticamente significativa (Mann-Whitney, $n=9$ vs. $n=9$, $p=0,0039$). O teste pareado por integrante (Wilcoxon, $n=3$ pares) não atingiu significância ($p=0,50$), o que é esperado dado o tamanho amostral reduzido nesse nível de agregação.

RQ2 — Taxa de sucesso nos testes

Hipótese informal: esperava-se uma taxa de sucesso levemente maior com IA.

Resultado: a hipótese não pôde ser testada estatisticamente, pois os 18 trials do grupo atingiram 100% de taxa de sucesso, com e sem IA, sem uma única exceção. A ausência completa de variância torna qualquer teste de diferença entre grupos matematicamente inaplicável. Esse resultado sugere que os katas escolhidos, dentro do time-box de 35 minutos, estavam calibrados a um nível de dificuldade resolvível de forma confiável em ambas as condições — uma limitação de desenho que impediu testar a RQ2 de fato.

RQ3 — Complexidade ciclomática e duplicação de código

Hipótese informal: não havia direção forte esperada.

Resultado: os dados mostram uma tendência de menor complexidade ciclomática bruta com IA (mediana 4,0 vs. 6,0 sem IA), mas essa diferença não atinge significância estatística (Mann-Whitney, $p=0,12$) com o tamanho amostral disponível ($n=9$ por grupo). A direção do efeito é consistente com a leitura de que o assistente tende a produzir soluções mais diretas — visível também no gráfico de dispersão da Figura 2, onde os pontos do tratamento IA se concentram no quadrante de menor tempo e menor complexidade bruta, mas a ausência de significância impede afirmar essa relação com confiança apenas com a complexidade bruta.

Um achado adicional, não previsto na hipótese original, emergiu ao examinar o volume de código produzido: a mediana de LOC foi maior com IA (30 linhas) do que sem IA (24 linhas), com dispersão bem mais ampla — três trials com IA ultrapassaram 70 linhas, acima de qualquer trial Manual (Figura 4). Isso sugere que o código gerado com assistência de IA tende a ser mais verboso. Quando a complexidade ciclomática é normalizada por esse volume (CC por mil linhas de código, Figura 5), a separação entre os tratamentos fica visualmente mais nítida do que na complexidade bruta — mediana de aproximadamente 200 CC/KLOC com IA contra 300 com Manual —, sugerindo que a ausência de significância na complexidade bruta pode ser parcialmente mascarada pela maior verbosidade do código gerado com IA. Em conjunto, essas duas visualizações complementares indicam uma narrativa mais matizada do que a complexidade bruta sozinha permite: o código com IA parece ser mais longo, porém proporcionalmente menos complexo por linha — uma distinção que a RQ3, tal como formulada no enunciado, não capturava isoladamente.

Não houve nenhuma duplicação de código detectada em nenhum dos 18 trials, com ou sem IA — resultado esperado dado o porte pequeno dos katas, que não permite avaliar essa dimensão da RQ3 de forma significativa neste laboratório.

RQ extra — Ambiguidade e desorganização de prompt

Foram desenhados três katas (k7, k8, k9) com regras de negócio deliberadamente omitidas do enunciado, apresentadas ao Gemini em registro informal e desorganizado. O padrão observado foi consistente e replicável: quando a regra oculta coincidia com uma convenção defensiva de programação (k7 — normalização de case), a IA implementou a solução correta de primeira tentativa em ambas as execuções (1 prompt), mesmo sem qualquer menção a isso no enunciado — o próprio código gerado trazia comentários evidenciando que a decisão veio de hábito de boa prática, não de inferência sobre o contexto. Em contraste, quando a regra oculta era uma política de negócio genuinamente arbitrária (k8 — carência de dias para professores; k9 — pontos em dobro às quartas), a IA necessitou de uma rodada adicional de correção em ambos os casos (2 prompts), mesmo com mensagens de erro deliberadamente vagas.

Notavelmente, a desorganização estrutural do prompt não parece ter sido o fator decisivo em nenhum dos três casos — os três foram escritos no mesmo registro informal, e ainda assim k8 e k9 exigiram correção enquanto k7 não, em ambas as execuções. Isso sugere que o fator determinante não é a ambiguidade do prompt em si, mas se a informação omitida é ou não inferível a partir de convenções gerais de engenharia de software — regras de negócio específicas de domínio geram fricção real na interação, enquanto convenções universais de código tendem a ser aplicadas por padrão, neutralizando o efeito de ambiguidade pretendido.

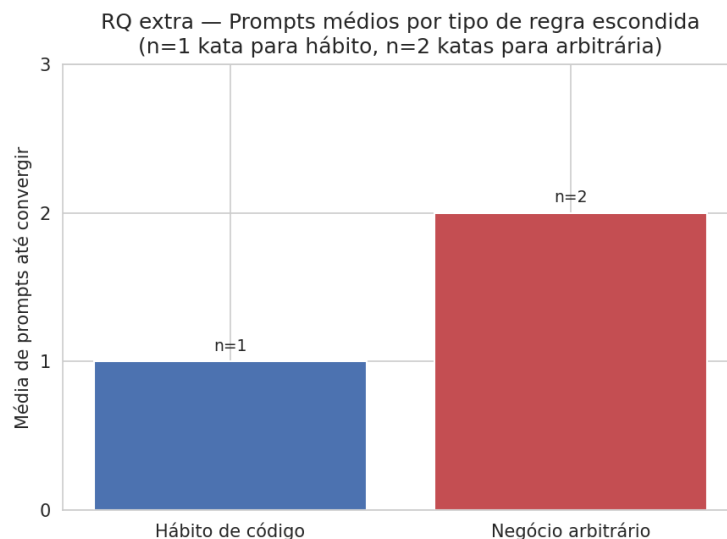


Figura 6 — Número médio de prompts até convergir, por tipo de regra escondida (hábito de código vs. negócio arbitrário).

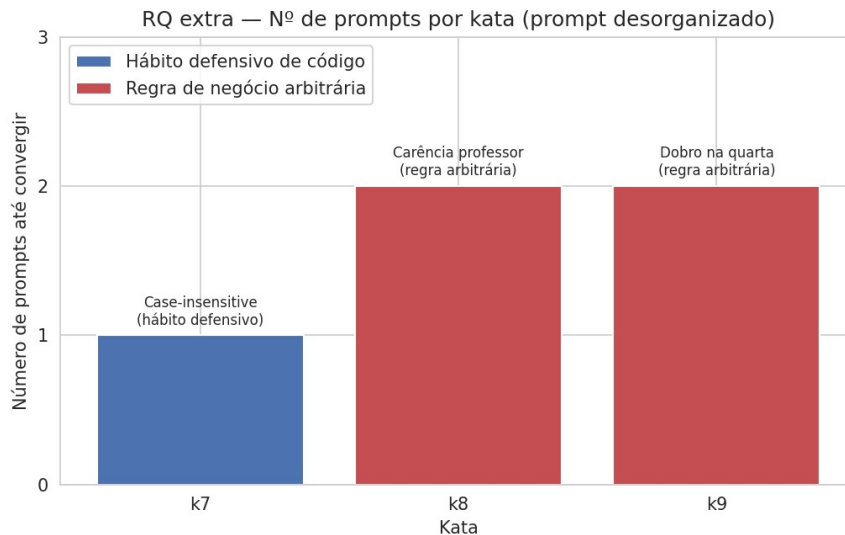


Figura 7 — Número de prompts até convergir, por kata individual (k7 replicado em duas execuções).

Limitações desta investigação: o tamanho amostral (3 katas, 4 trials) é pequeno demais para teste estatístico formal — a conclusão é qualitativa e descritiva. As mensagens de erro repassadas à IA, embora deliberadamente vagas, ainda carregavam alguma informação residual que pode ter facilitado a correção mais do que um cenário de feedback totalmente opaco permitiria.

5. Conclusão

Este laboratório conduziu um experimento controlado, within-subject e contrabalançado, comparando o uso de um assistente de IA generativa (Claude) contra a codificação manual na resolução de seis katas de programação, com três integrantes atuando simultaneamente como pesquisadores e sujeitos experimentais. O achado central, sustentado por evidência estatística robusta, é que o assistente de IA reduz substancialmente o tempo de resolução de tarefas de programação de pequeno porte, sem custo aparente em taxa de sucesso — resultado que confirma a hipótese informal mais forte do grupo e representa a contribuição mais defensável deste trabalho.

As demais questões de pesquisa produziram resultados mais matizados. A taxa de sucesso não pôde ser comparada estatisticamente entre tratamentos, já que todos os 18 trials atingiram 100% de aproveitamento — um resultado que revela uma limitação real do desenho, não pretendida originalmente. A complexidade ciclomática mostrou uma tendência de redução com uso de IA, mas sem significância estatística no tamanho amostral disponível, e a duplicação de código não pôde ser avaliada por ausência completa de ocorrências nos katas escolhidos.

A inovação proposta pelo grupo produziu um achado qualitativo claro e replicável, ainda que baseado em amostra pequena: regras que coincidem com hábitos defensivos de programação são aplicadas por padrão, independentemente de estarem explícitas no enunciado, enquanto regras de negócio genuinamente arbitrárias geram necessidade real de correção iterativa. Esse resultado nuança a hipótese original de forma que os próprios autores não haviam antecipado, e ilustra como um desenho experimental inicial pode precisar de ajuste metodológico durante a própria execução.

Limitações

O tamanho amostral do experimento principal (n=9 por tratamento) é pequeno para os padrões de significância estatística convencionais, o que provavelmente explica por que RQ3 não atingiu significância apesar de mostrar uma tendência consistente. A calibração de dificuldade dos katas, embora bem-sucedida para RQ1, foi excessivamente permissiva para RQ2 — nenhum trial falhou, eliminando a variância necessária. Um erro de rotulagem de dados foi identificado e corrigido durante a consolidação, reforçando a importância de validação cruzada em coleta distribuída entre múltiplos integrantes. A investigação de inovação teve amostra ainda menor e não permite generalização além do padrão qualitativo observado.

Trabalhos futuros

Com mais tempo, o grupo teria calibrado os katas para uma dificuldade ligeiramente maior, de forma a gerar variância real em taxa de sucesso e permitir testar RQ2 de fato. A investigação de ambiguidade de prompt, que se mostrou promissora, mereceria ser conduzida com um número maior de katas por categoria e com um protocolo mais rigoroso de controle sobre a informação vazada nas mensagens de erro repassadas à IA — um refinamento natural para o próximo laboratório da disciplina.

6. Referências

- BASILI, Victor R.; CALDIERA, Gianluigi; ROMBACH, H. Dieter. The Goal Question Metric Approach. In: Encyclopedia of Software Engineering. Wiley, 1994.
- GITHUB. GraphQL API documentation. Disponível em: <https://docs.github.com/en/graphql>
- GITHUB. GitHub Projects documentation. Disponível em: <https://docs.github.com/en/issues/planning-and-tracking-with-projects>
- RADON. Radon documentation — Code metrics for Python. Disponível em: <https://radon.readthedocs.io/>
- JSCPD. jscpd — Copy/paste detector for programming source code. Disponível em: <https://github.com/kucherenko/jscpd>
- PYTEST. pytest documentation. Disponível em: <https://docs.pytest.org/>
- VIRTANEN, Pauli et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. Nature Methods, v. 17, p. 261–272, 2020.